Architecture and Workflow

Compile-time pipeline, bytecode layout, VM state, and execution traces

CVM++ runs in two phases: *compile time* (lexer → parser → compiler) and *runtime* (stack VM). `compile_frontend()` in `compile.cpp` produces a `BytecodeChunk`; `main.cpp` calls `execute()` unless `-c` (compile-only) is set.

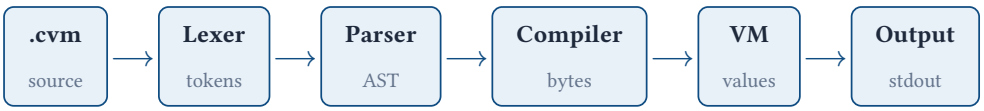

Figure 1: Pipeline: source → tokens → AST → bytecode → VM.

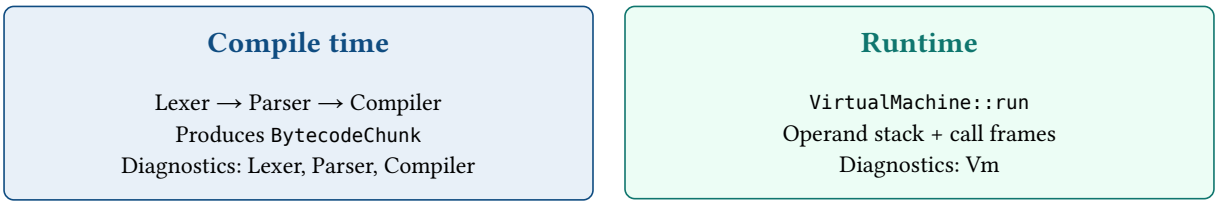

Figure 2: Compile time (front end) versus runtime (virtual machine).

```
main.cpp
  → compile_frontend(source)
       → Lexer::tokenize()      → vector<Token>
       → Parser::parse()        → Program (AST)
       → Compiler::compile()    → BytecodeChunk
  → VirtualMachine::run(chunk) → stdout + diagnostics
```

Stage	Input	Output	Source
Lexer	Source string	`vector<Token>`	`lexer.cpp`
Parser	Tokens	`Program` AST	`parser.cpp`
Compiler	AST	`BytecodeChunk`	`compiler.cpp`
VM	Bytecode	Output + diagnostics	`vm.cpp`

Key terms

`FrontEndResult`	One run's bundle: source text, lexer/parser/compiler results, optional VM output, and a token copy for `-d` debug dumps.
`Program`	AST root: `functions[]` (top-level `fn` declarations) plus `statements[]` (main script body).
`BytecodeChunk`	`code` byte vector, `names[]` global pool, and `functions[]` metadata (entry address, arity).
Jump patching	Forward branches emit a placeholder u32; `patch_u32` writes the real offset when the target is known.
`CallFrame`	`return_ip` plus `locals[]`; pushed on `CALL`, popped on `RETURN` with the result on the stack.
`VmSession`	REPL-only `unordered_map` of global name → value; file runs use indexed `globals_` instead.
Failsafe	Runtime guard (divide by zero, stack limits, uninitialized load, type errors) → diagnostic, exit 2.

`compile_frontend` walkthrough

`FrontEndResult` (`compile.hpp`) stops early on failure: lexer errors skip parse; parse errors skip compile. The VM is **not** run inside `compile_frontend` — `main` calls `execute()` separately so `-c` and `:disasm` share the same path.

1. Lexer fills `result.lex`; on failure, diagnostics copied to parse bag and return.
2. Parser moves tokens, builds `Program`; lexer warnings merged into parse diagnostics.
3. Compiler lowers AST; undefined callee or wrong arity → `Phase::Compiler`.
4. `execute(chunk, …)` runs only when compile succeeded and not `compile_only`.

Lexer

Scans source left-to-right: skips whitespace and `//` comments; emits tokens until `Eof`.

Keywords: `let`, `fn`, `return`, `if`, `else`, `while`, `print`, `input`, `true`, `false`.

Multi-char operators: ==, !=, <=, >=, plus single-char `+ - * / ( ) { } ; , = < >`.

Example — source `let x = 42;` yields (conceptually):

#	Type	Lexeme
1	`Let`	`let`
2	`Identifier`	`x`
3	`Assign`	`=`
4	`Integer`	`42`
5	`Semicolon`	`;`
6	`Eof`	(end)

Failures (`Phase::Lexer`): invalid character, integer overflow, null byte in source.

Parser

Produces `Program` with two regions compiled differently:

1. `functions` — every top-level `fn` (emitted **before** main in bytecode).
2. `statements` — main script body (runs after entry `JUMP` lands here).

Precedence (low → high): == / != → < > <= >= → + - → * / → unary - → primary (literal, call, `input`, parens).

AST nodes: `IntLiteralExpr`, `BoolLiteralExpr`, `VariableExpr`, `BinaryExpr`, `UnaryExpr`, `CallExpr`, `InputExpr`; `LetStmt`, `AssignStmt`, `PrintStmt`, `ReturnStmt`, `IfStmt`, `WhileStmt`, `BlockStmt`; `FunctionDecl`.

Example — `print 1 + 2;` parses as `PrintStmt` → `BinaryExpr(+)` → two `IntLiteralExpr`.

Failures (`Phase::Parser`): unexpected token, `10 = x;` (invalid assign target), unclosed `(` / `{`. Recovery: `synchronize()` to next `;` or `}`.

Compiler

Figure 3: Logical layout of `BytecodeChunk::code` after compilation (function-first placement).

Emission order (`compile_program`)

```
1. emit JUMP + placeholder u32
2. for each fn in program.functions: compile_function
3. main_entry = current offset; patch JUMP → main_entry
4. for each stmt in program.statements: compile_statement
5. emit HALT
```

Without step 1, `ip` starts inside the first function ⟶ `LOAD_LOCAL outside of function` at runtime.

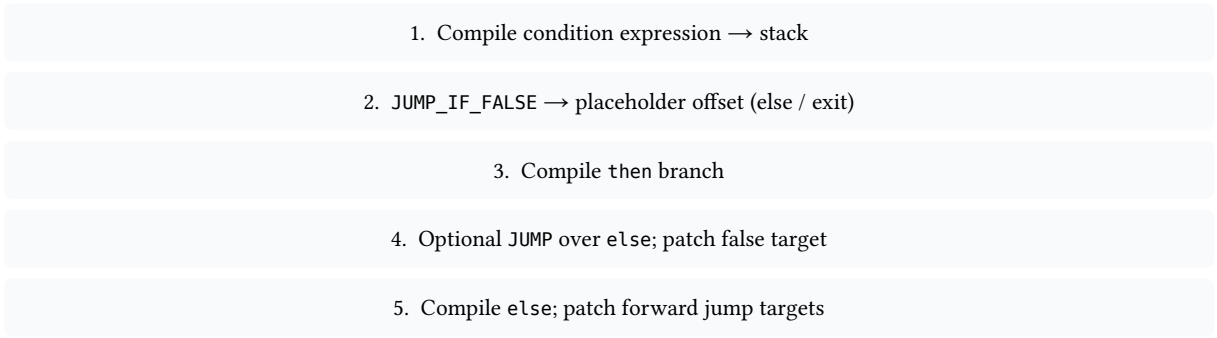

Figure 4: `if` lowering: condition compiled first; false branch jumps around `then` or to `else`.

Control-flow lowering

1. **`if` / `else`:** condition ⟶ `JUMP_IF_FALSE` (patch to else/exit) ⟶ then ⟶ optional `JUMP` over else ⟶ patch targets.
2. **`while`:** loop label ⟶ condition ⟶ `JUMP_IF_FALSE` exit ⟶ body ⟶ `JUMP` to loop start ⟶ patch exit.
3. **Calls:** compile args left-to-right ⟶ `CALL` u16 fn index, u8 argc.

Names

Globals: intern in `chunk.names`, `LOAD_VAR` / `STORE_VAR` with u16 index. Locals: slots 0…n−1 in active frame; `LOAD_LOCAL` / `STORE_LOCAL` with u8 slot.

```
fn factorial(n) {
  if (n <= 1) { return 1; }
  return n * factorial(n - 1);
}
print factorial(5);
```

Expected stdout: `120`. Compiler records `FunctionMeta` (name, entry address, arity) before main statements.

Virtual machine

State

Field	Role
`ip_`	Index into `chunk_.code`
`stack_`	Operand stack (`VmValue` = `int64_t` | `bool`)
`globals_` / `global_init_`	File-mode globals by index
`frames_`	Call stack of `CallFrame`
`session_`	Optional REPL `unordered_map` for persistent globals

Execution: read opcode byte ⟶ dispatch. Binary ops pop **right** then **left**. Limits: stack depth 65536; 1M steps per run (infinite-loop guard).

Failures (`Phase::Vm`): divide by zero, stack under/overflow, uninitialized global, type mismatch (`true + 5`), bad `RETURN`, `LOAD_LOCAL` outside function, IP past end of bytecode. File mode: exit code **2**.

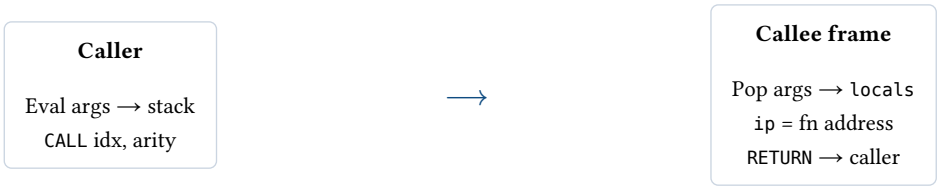

Figure 5: Function call: arguments on operand stack become callee local slots.

Opcode reference

Opcode	Hex	Operands	Effect
`PUSH_INT`	0x01	i64	Push integer literal
`PUSH_BOOL`	0x02	u8	Push boolean
`LOAD_VAR` / `STORE_VAR`	0x10 / 0x11	u16	Global by name index
`LOAD_LOCAL` / `STORE_LOCAL`	0x12 / 0x13	u8	Frame local slot
`ADD`…`GE`, `NEG`	0x20–0x2A	—	Arithmetic / compare; pop b, then a
`INPUT` / `PRINT`	0x30 / 0x31	—	Read stdin line / pop to output
`JUMP` / `JUMP_IF_FALSE`	0x40 / 0x41	u32 offset	Branch; false pops bool
`CALL` / `RETURN`	0x42 / 0x43	u16 idx, u8 argc / —	Invoke / return with value
`HALT`	0xFF	—	Stop VM loop

Execution traces

Trace A — `print 1 + 2;`

Step	Instruction	Stack after
1	`PUSH_INT 1`	[1]

2	`PUSH_INT 2`	[1, 2]
3	`ADD`	[3]
4	`PRINT`	stdout: 3

Trace B — `let x = 3; print x;`

Step	Effect	Stack
1	`PUSH_INT 3`	[3]
2	`STORE_VAR x`	[]
3	`LOAD_VAR x`	[3]
4	`PRINT`	output 3

Trace C — recursive `factorial(5)`

Narrative: VM starts at patched main (not inside `factorial`). Main pushes `5`, `CALL` → frame with `n=5`. Each call evaluates `n <= 1`; recursion until `n=1` returns `1`; unwinding multiplies to **120**; main `PRINT` shows `120`.

Entry points and debug

Mode	Command	Behavior
File	`cvmpp script.cvm`	Fresh globals each run
REPL	`cvmpp`	`VmSession` keeps variables between lines
Debug	`-d`	Token table → AST → bytecode → runtime output
Compile-only	`-c`	Front end only; no VM side effects

> **Debug workflow**
>
> Run `./build/cvmpp -d examples/hello.cvm` and read output in order: **tokens → AST → bytecode → runtime**. In REPL, `:disasm` on a path (or last chunk) uses the same compile path without executing `print` / `input`.

Diagnostics and exit codes

Phase	Source	Typical failure
`Lexer`	`lexer.cpp`	Bad character, integer overflow
`Parser`	`parser.cpp`	Syntax, invalid assignment target
`Compiler`	`compiler.cpp`	Undefined function, wrong arity
`Vm`	`vm.cpp`	Div0, stack, uninitialized var, type error
`Repl`	`main.cpp`	Unknown command, missing file

Exit	Meaning
0	Success
1	Lexer, parser, or compiler error
2	VM runtime error

Module dependencies

```
main.cpp → compile.cpp → lexer, parser, compiler, vm, ui
compiler.cpp, vm.cpp → bytecode.hpp, opcode.hpp
parser.cpp → ast.hpp
```

All modules link into one `cvmpp` binary — no dynamic plugins. Compiler and VM share `BytecodeChunk` layout in `bytecode.hpp`.